

Trajectory-guided Control Prediction for End-to-end Autonomous Driving: A Simple yet Strong Baseline

2022 NeurIPS(Neural Information Processing Systems)

Background

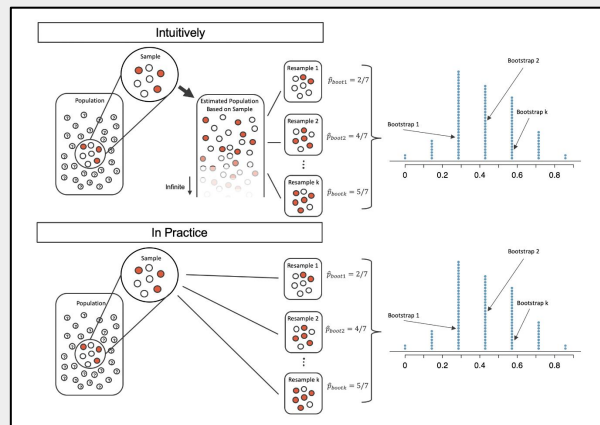
Reinforcement learning(RL)

: A promising way to address the problem of being more robust to the distribution shifts of datasets

- Bootstrap / Dynamic programming / Monte-Carlo method

Bootstrap * Estimation based

: It is possible to estimate the distribution or confidence interval of a parameter with just one given data sample



Dynamic programming(ex. Value Iteration)

: An Algorithm applied to a structure where the solution to a larger problem contains the solution to a smaller problem.

$$V(s) \leftarrow \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') \right]$$

* Estimation based
(Bellman Equation)

[Optimal substructure] The value of the current state is updated by the values of the next states

[Overlapping subproblems] The update is performed for all states repeatedly

Monte-Carlo method * Observation based

$$V(s) \leftarrow V(s) + \alpha(G_t - V(s)) \quad \text{where } G_t = r_t + \gamma r_{t+1} + \dots + \gamma^{T-t} r_T$$

: Without requiring a model, the agent performs episode-based simulations and updates the value function using the **actual total return**

Background

Reinforcement learning(RL)

: A promising way to address the problem of being more robust to the distribution shifts of datasets.

- TD(Temporal difference)-Learning → State-Value based on given policy, **on-policy**

$$V(s_t) \leftarrow V(s_t) + \alpha (r_t + \gamma V(s_{t+1}) - V(s_t))$$

* Observation

* Estimation - TD Target; prediction of future reward

$$V(s) \leftarrow V(s) + \alpha (G_t - V(s)) \quad \text{where } G_t = r_t + \gamma r_{t+1} + \dots + \gamma^{T-t} r_T$$

Monte-Carlo Method

$$V(s) \leftarrow \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') \right]$$

Dynamic programming

- Q-Learning → State-Action Value without policy, **off-policy**)

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_t + \underbrace{\gamma \max_{a'} Q(s_{t+1}, a')}_{\text{TD Target (Bellman equation)}} - Q(s_t, a_t) \right)$$

Update Q-Table(2D space)

* State-Action

Background

Reinforcement learning(RL)

: A promising way to address the problem of being more robust to the distribution shifts of datasets.

- DQN(Deep Q-Network) / DDPG(Deep Deterministic Policy Gradient)

DQN

: Q-table is infeasible for large or continuous state space

→ $Q(s, a; \theta) \approx$ Neural Network with parameters θ
(Function Approximation : unable to directly update Q)

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

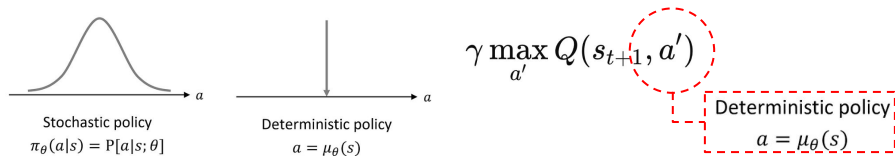
$$L(\theta) = (r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-) - Q(s_t, a_t; \theta))^2$$

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta) \quad \text{* Parameter update}$$

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

DDPG

: DQN is infeasible for continuous action space



Policy Gradient

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{s \sim d^{\pi}, a \sim \pi} [\nabla_{\theta} \log \pi(a|s) Q^{\pi}(s, a)]$$

* Based on stochastic policy $\pi(a|s)$

Deterministic Policy Gradient

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{s \sim D} [\nabla_a Q(s, a)|_{a=\mu(s)} \cdot \nabla_{\theta} \mu_{\theta}(s)]$$

* Based on deterministic policy $\mu_{\theta}(s)$

$$Q^{\mu}(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma Q^{\mu}(s_{t+1}, \mu(s_{t+1}))]$$

$\mu(s; \theta^{\mu}) \rightarrow$ Actor network

$Q(s, a; \theta^Q) \rightarrow$ Critic network

$$\mu : \mathcal{S} \leftarrow \mathcal{A}$$

Background

Reinforcement learning(RL)

: A promising way to address the problem of being more robust to the distribution shifts of datasets.

- Rainbow DQN(Extended DQN)

Double DQN

$$\operatorname{argmax}_{a'} q_{\theta}(S_{t+1}, a')$$

: Addresses overestimation of DQN by decoupling the selection of the action from its evaluation in the maximization performed for the bootstrap target.

$$L(\theta) = (R_{t+1} + \gamma \max_{a'} q_{\bar{\theta}}(S_{t+1}, a') - q_{\theta}(S_t, A_t))^2$$

* TD Target

$$y_t = r_t + \gamma Q(s_{t+1}, \arg \max_a Q(s_{t+1}, a; \theta); \bar{\theta}^-)$$

* Online Network

* Target network

Prioritized replay

: PER(Prioritized *experience* replay) samples transitions with probability relative to the last absolute TD error

$$p_t \propto |R_{t+1} + \gamma \max_{a'} q_{\bar{\theta}}(S_{t+1}, a') - q_{\theta}(S_t, A_t)|^{\omega}$$

$$P(t) = \frac{p_t}{\sum_k p_k} \quad w_t = \left(\frac{1}{N \cdot P(i)} \right)^{\beta}$$

* Prioritize w/ replay buffer

* Weight for Loss function

$$\rightarrow L(\theta) = \mathbb{E}_{i \sim P(i)} [w_i (y_i - Q(s_i, a_i; \theta))^2]$$

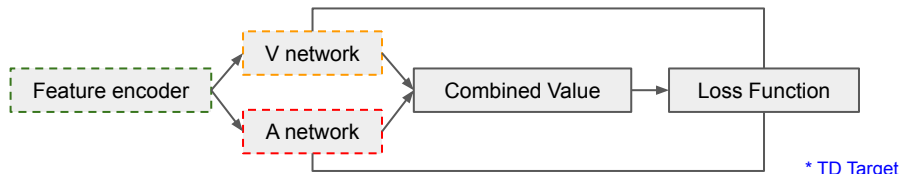
Dueling Network

: Estimates state value(V) and action advantage(A) separately and combines them to compute Q-value: $Q(s, a) = V(s) + A(s, a) - \text{mean}_a A(s, a')$

: Enables more stable learning and better distinction between state value and action effects, especially when many actions have similar values

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right) = \sum_a A(s, a) - \sum_a \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') = \sum_a A(s, a) - |\mathcal{A}| \cdot \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') = 0$$

$$q_{\theta}(s, a) = v_{\eta}(f_{\xi}(s)) + a_{\psi}(f_{\xi}(s), a) - \frac{1}{N_{\text{actions}}} \sum_{a'} a_{\psi}(f_{\xi}(s), a')$$



$$L(\theta) = (r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \bar{\theta}^-) - Q(s_t, a_t; \theta))^2$$

$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta)$ * Parameter update

Background

Reinforcement learning(RL)

: A promising way to address the problem of being more robust to the distribution shifts of datasets.

- Rainbow DQN(Extended DQN)

Multi-step Learning

: Uses forward-view multi-step targets to accelerate learning

: n-step return from a given state is defined as:

$$R_t^{(n)} \equiv \sum_{k=0}^{n-1} \gamma_t^{(k)} R_{t+k+1}$$

: The loss function becomes:

$$L(\theta) = (R_t^{(n)} + \gamma_t^{(n)} \max_{a'} q_{\theta}(S_{t+n}, a') - q_{\theta}(S_t, A_t))^2$$

* TD Target

Noisy Net

: Injects noise into the weights, enabling efficient exploration.

: Built-in stochasticity is especially beneficial in sparse reward environments, where effective exploration is critical.

$$y = \underbrace{b + Wx}_{\text{deterministic stream}} + \underbrace{b_{\text{noisy}} \circ \epsilon_b + (W_{\text{noisy}} \circ \epsilon_w)x}_{\text{stochastic stream}}$$

Distributional RL

: Learns the full distribution of possible returns(not just their expected value) by modeling the return as a random variable, thereby improving decision-making through better representation of uncertainty and variability in outcomes.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

$$Z(s, a) \sim \text{distribution of returns} \quad \leftarrow \quad G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

$$z_i = v_{\min} + (i - 1) \cdot \frac{v_{\max} - v_{\min}}{N_{\text{atoms}} - 1}, \quad i = 1, \dots, N_{\text{atoms}}$$

$$d(s, a) = (z, p_{\theta}(s, a)) \rightarrow \text{categorical distribution}$$

$$Q(s, a) = \mathbb{E}_{Z \sim d(s, a)}[Z] = \sum_{i=1}^{N_{\text{atoms}}} z_i \cdot p_i \quad \leftarrow \quad Q(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q(s', a')]$$

$$a_{t+1}^* = \arg \max_a \mathbb{E}_{Z \sim d(s_{t+1}, a)}[Z] = \arg \max_a \sum_{i=1}^{N_{\text{atoms}}} z_i \cdot p_{\theta}^i(s_{t+1}, a)$$

$$d'_t \equiv (R_{t+1} + \gamma z, p_{\theta}(s_{t+1}, a_{t+1}^*))$$

$$\text{L2 Projection: } \Phi_z d'_t, \quad \text{KL Divergence: } \text{Loss} = D_{KL}(\Phi_z d'_t \parallel d_t)$$

Background

Reinforcement learning(RL)

: A promising way to address the problem of being more robust to the distribution shifts of datasets.

- Rainbow DQN(Extended DQN)

The Integrated Agent

Multi-step & distributional RL & Double DQN

$$d_t^{(n)} = (R_t^{(n)} + \gamma_t^{(n)} z, \overset{* \text{ Target network}}{p_{\bar{\theta}}(s_{t+n}, \overset{* \text{ Online network}}{a_{t+n}^*})})$$

* Separate action and estimation

Prioritize Experience Replay

$$p_t \propto \left(D_{\text{KL}}(\Phi_z d_t^{(n)} || d_t) \right)^\omega \quad P(t) = \frac{p_t}{\sum_k p_k} \quad w_t = \left(\frac{1}{N \cdot P(i)} \right)^\beta$$

Dueling DQN

$$p_\theta^i(s, a) = \frac{\exp(v_\eta^i(\phi) + a_\psi^i(\phi, a) - \bar{a}_\psi^i(s))}{\sum_j \exp(v_\eta^j(\phi) + a_\psi^j(\phi, a) - \bar{a}_\psi^j(s))} \quad \phi = f_\xi(s) \text{ and } \bar{a}_\psi^i(s) = \frac{1}{N_{\text{actions}}} \sum_{a'} a_\psi^i(\phi, a')$$

+ replace all linear layers with

$$y = \underbrace{b + Wx}_{\text{deterministic stream}} + \underbrace{b_{\text{noisy}} \circ \epsilon_b + (W_{\text{noisy}} \circ \epsilon_w)x}_{\text{stochastic stream}}$$

 $\mathcal{L}(\theta) = D_{\text{KL}} \left(\Phi_z \left[\left(R_t^{(n)} + \gamma^n z, p_{\theta^-}(s_{t+n}, a_{t+n}^*) \right) \right] \parallel (z, p_\theta(s_t, a_t)) \right)$

Introduction

Trajectory-based vs Control-based Autonomous driving

: The output of end-to-end autonomous driving generally falls into two forms: **trajectory/waypoints** and **direct control actions**

- Trajectory-based prediction

[Pros] Considers a longer time horizon, easy to integrate with other modules (e.g., multi-agent prediction, semantic or occupancy prediction)

[Cons] Converting trajectory into control actions is non-trivial, difficult to follow planned trajectory accurately

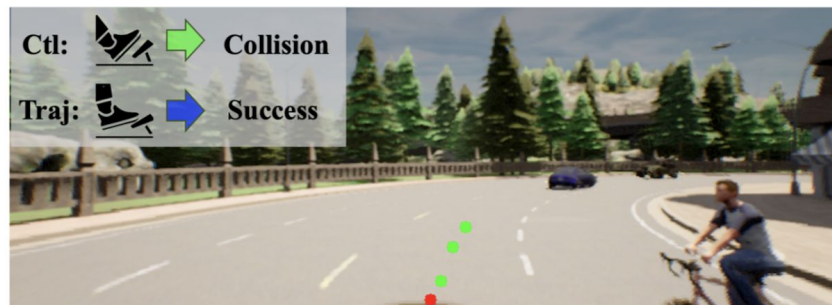
- Control-based prediction

[Pros] Focuses only on the current state(may react too late to avoid collisions)

[Cons] Predictions across time steps can be inconsistent which leads to unstable or jerky driving



(a)



(b)

Figure 1: Typical failure cases of two prediction paradigms. Red dots indicate the trajectory prediction, blue dots are the actual path following the red trajectory with PID controllers, and green dots denote the actual path from control-based method. (a) trajectory-based methods may struggle for big turns. (b) control-based methods may have a reaction latency and suffer from abrupt obstacles due to focusing on current time step only. These observations motivate us to propose a unified framework to combine these two worlds for mutual benefits.

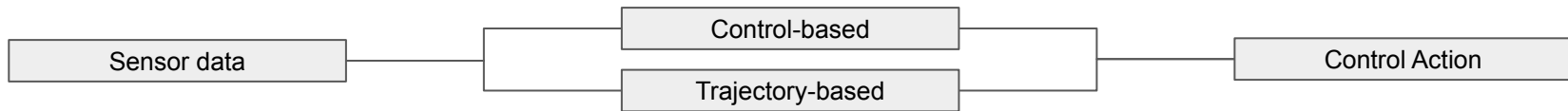


How to combine these two forms of prediction model as well as their outputs?

Introduction

Trajectory-based + Control-based Autonomous driving

: Only uses a monocular camera, TCP achieves a 75.137 driving score and ranks 1st on the public CARLA Leaderboard, even surpassing prior state-of-the-art methods using multiple cameras and a LiDAR by 13.291 points.



- Combine outputs directly (Naive approach)

Train a control prediction model and a trajectory planning model separately, and combine outputs

: Not only doubles the size of the model, but also ignores possible useful correlations between two forms.

- TCP (Trajectory-guided Control Prediction)

[Shared Backbone]

Extracts common visual and contextual features, reducing computational cost and improving generalization.

[Multi-step Control Prediction]

Predicts a sequence of future control actions, enabling temporal consistency and long-term reasoning.

- Traditional behavior cloning often focuses only on the current time step, assuming *each state-action pair is independent and identically distributed*.

This IID assumption is *inaccurate for sequential decision-making tasks like driving* and can degrade long-term performance.

- Temporal Module required (GRU) → Models interactions between the ego-agent and the environment (e.g., other vehicles, traffic lights).

[Trajectory-guided Attention Mechanism]

Predicted trajectory guides the control branch's attention toward relevant visual regions.

[Situation-based Fusion Scheme]

Forming a **self-ensemble** that dynamically combines outputs from the two branches based on driving context(dynamic environmental factors).

Related Work

End-to-end Autonomous Driving: Hybrid Learning framework

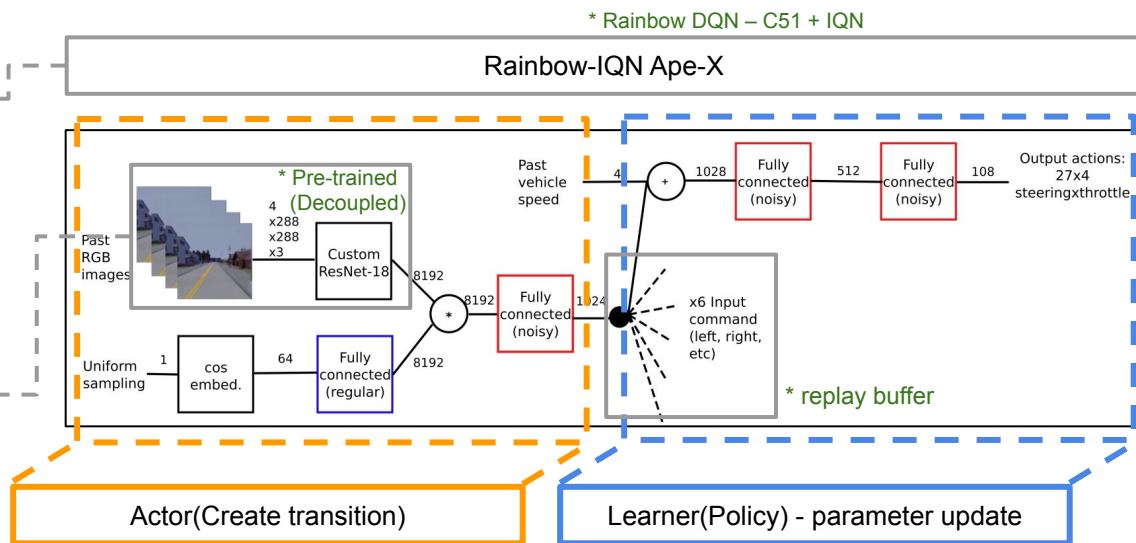
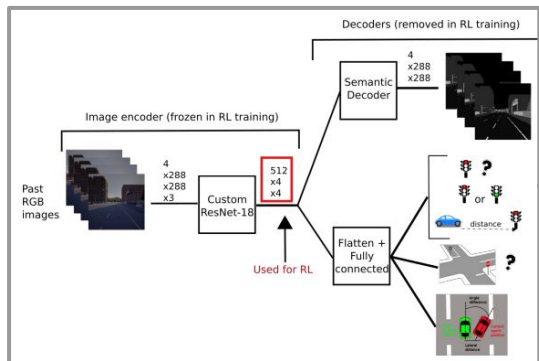
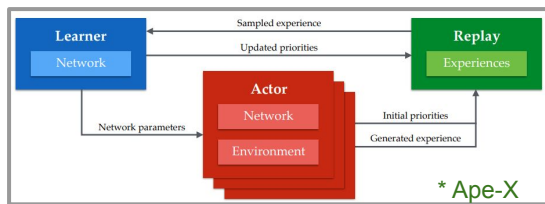
: End-to-End Model-Free Reinforcement Learning for Urban Driving using Implicit Affordances [CVPR, 2020]

- **Separates policy and transition sampling module with decoupled perception task** that samples environment dynamics

- Rainbow-IQN based Ape-X

- Actor : Update replay buffer of Learner w/o PER / Inference only(no parameter update), use parameters of learner

- Learner : Update parameters with transitions from prioritized experience replay buffer



Related Work

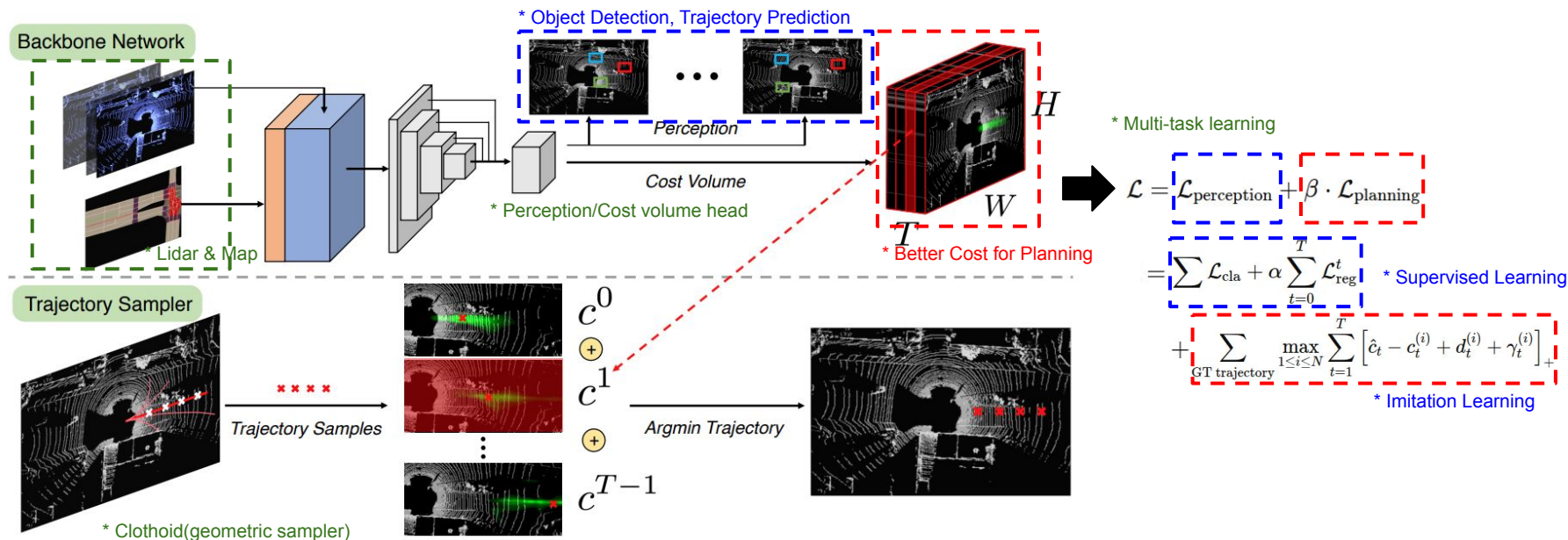
End-to-end Autonomous Driving: Hybrid Learning framework

: End-to-end interpretable neural motion planner [CVPR, 2019]

- (Inductive bias) The **trajectory branch provides motion context**, helping **guide the cost volume head** for more informed and interpretable planning outputs.

Trajectory prediction branch : Predicts future motion of surrounding objects.

Planning branch : Uses a learned cost volume to classify and select optimal ego-vehicle trajectories.

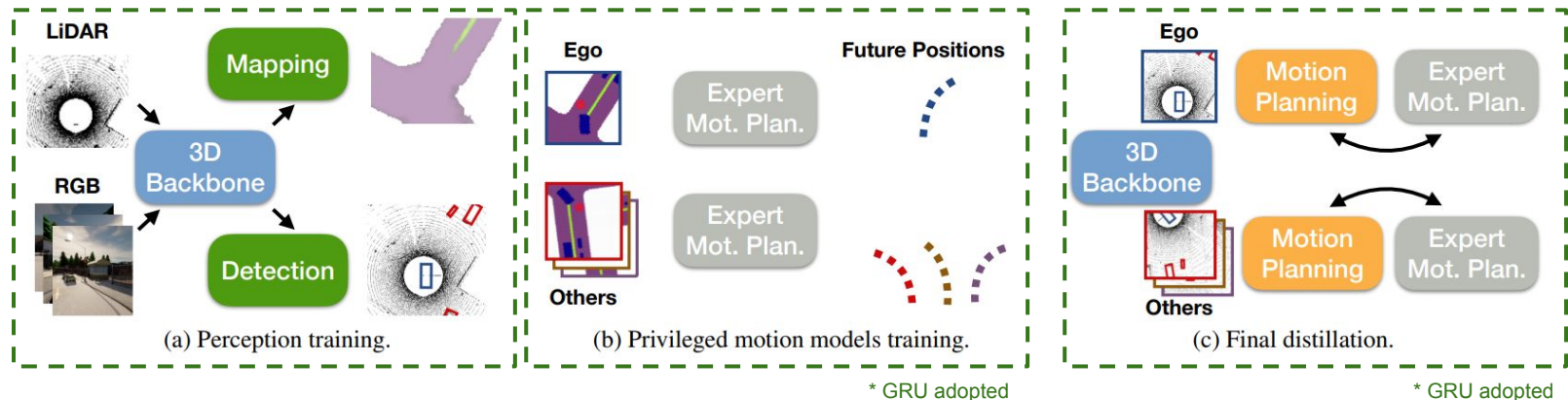


Related Work

End-to-end Autonomous Driving: GRU module adopted & Multi-Task Trajectory prediction

: TransFuser: Imitation with Transformer-Based Sensor Fusion for Autonomous Driving [CVPR, 2021]/[PAMI, 2022]

: LAV: Learning from All Vehicles [CVPR, 2022]



1. Independently train perception and trajectory prediction models:

- **a) 3D Perception Backbone** (3D object detection and semantic map prediction)
 - : supervised learning using object detection and semantic map labels to learn **viewpoint-invariant 3D spatial features**.
- **b) Expert Motion Planner** (Trajectory prediction model)
 - : Trained to predict the **future trajectories of all vehicles** using **ground-truth 3D bounding boxes** as input.

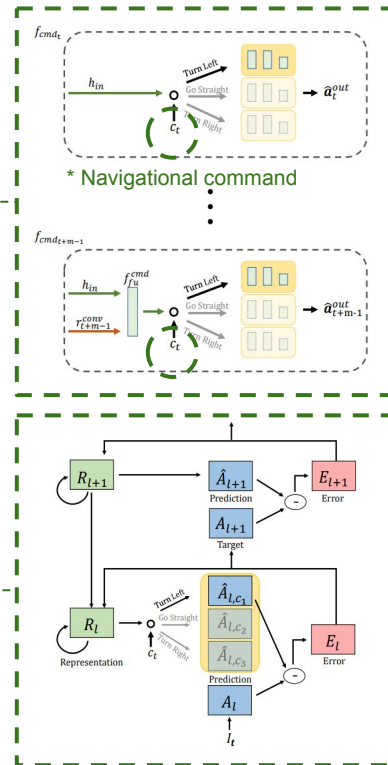
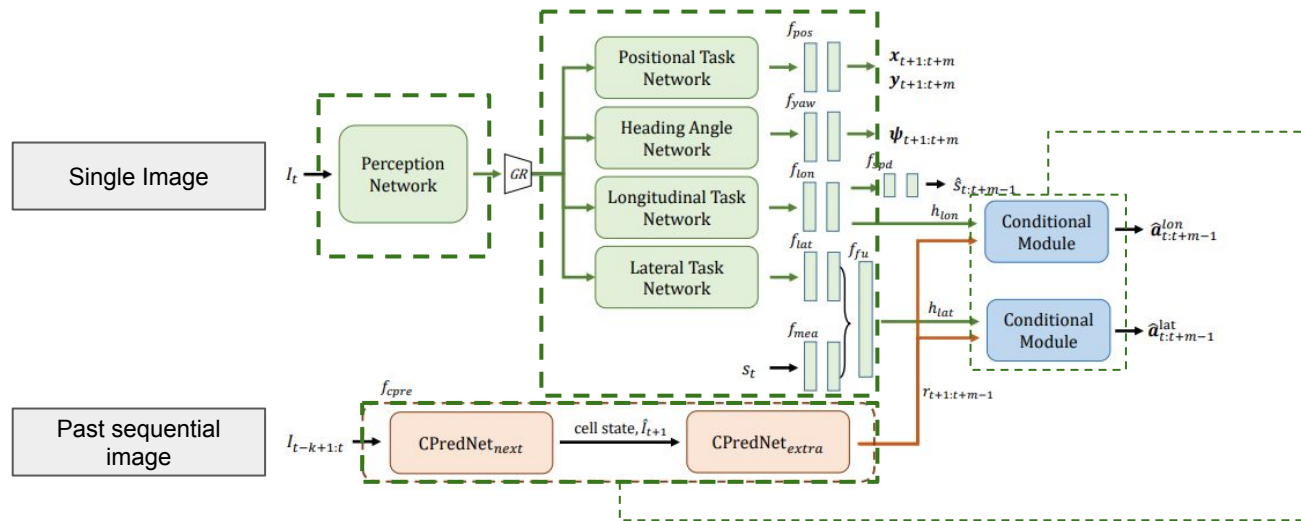
2. Distillation Training:

- **c) An end-to-end student model** is trained using the outputs of (a) and (b). The distillation process involves both **feature-level** and **output-level** supervision(multi-task learning)

Related Work

End-to-end Autonomous Driving: Multi-Task Control prediction

: FASNet: Multi-task Learning with Future States for Vision-based Autonomous Driving [ACCV, 2020]



Single image branch : Perception + Multi-task Control Prediction

- Perception : Images $\rightarrow$ latent features, shared by localization and control branches
- Multi-task : Localization - position, heading angle / Control - Longitudinal(throttle, brake), Lateral(steering)

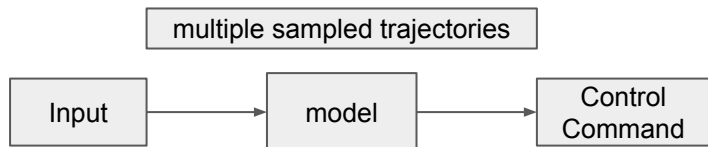
Sequence Image branch : future-aware representation(Traj like)

- CpredNet : Temporal image sequences are transformed into control feature representations

Related Work

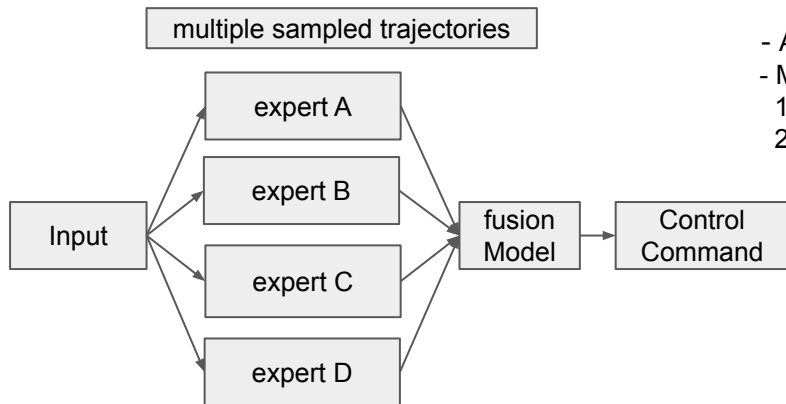
End-to-end Autonomous Driving: Two Classic Ensemble methods

: Test-Time Augmentation(TTA); Test time



- A single model predicts control outputs conditioned on multiple sampled trajectories during inference
- Method
 1. Sample multiple candidate trajectories.
 2. Predict control for each one.
 3. Aggregate the results (e.g., averaging, selecting minimum cost)

: Fusion of experts; Training time

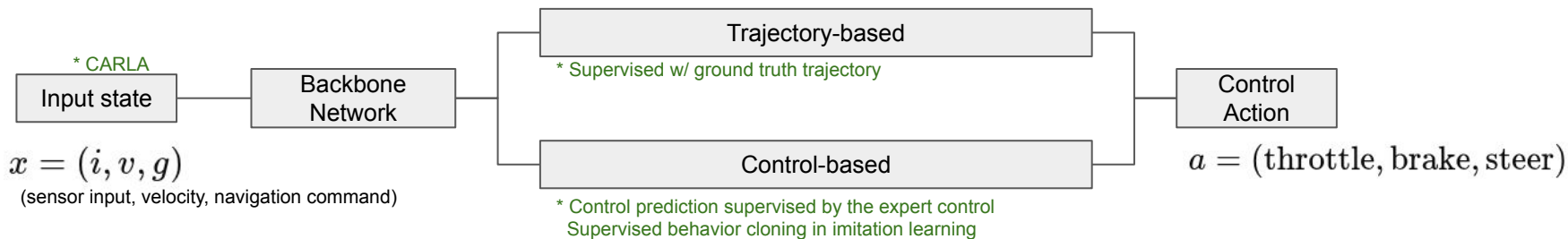


- An ensemble of expert networks, each trained on different trajectory conditions
- Method
 1. Each expert predicts control for a specific trajectory
 2. Fuse predictions using weighting, voting, or selection strategies

Method

TCP(Trajectory-guided Control Prediction)

: Problem Formulation



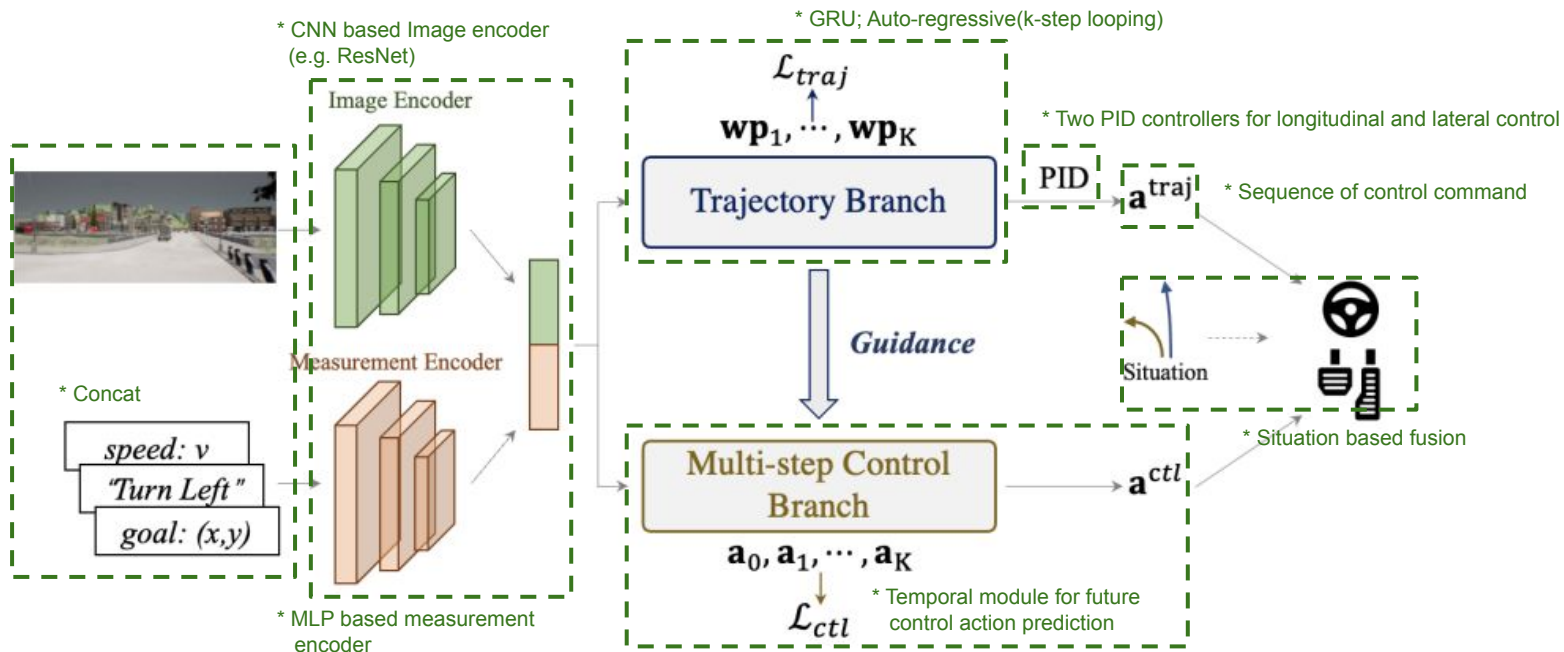
* Label (Supervision Signal)
: Expert Demonstration(Roach)

- A learning-based expert trained with RL using **privileged information** (roads, lanes, traffic elements) rendered in **2D BEV**.
- Compared to rule-based experts, it provides **richer supervision** beyond direct actions.
- Two additional objectives guide the student model
 - : **Feature loss**: Aligns intermediate features with the expert.
 - : **Value loss**: Predicts expected return as auxiliary signal.

Method

TCP(Trajectory-guided Control Prediction)

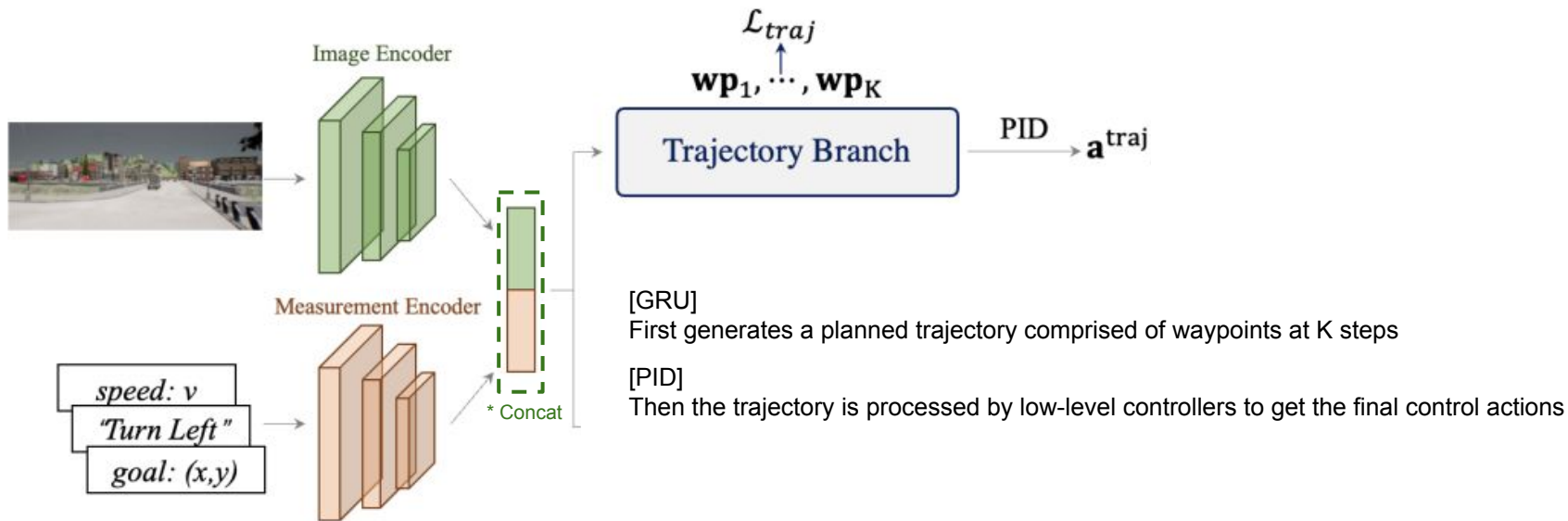
: Architecture Overview



Method

TCP(Trajectory-guided Control Prediction)

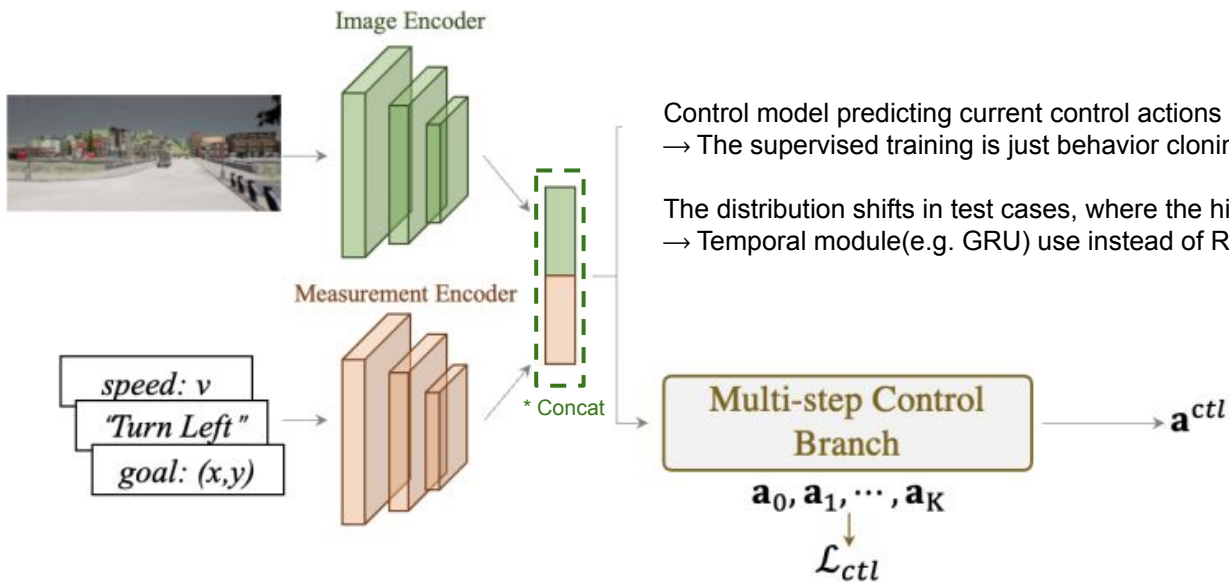
: Trajectory Branch



Method

TCP(Trajectory-guided Control Prediction)

: Multi-Step Control Branch



Control model predicting current control actions based on current input only,
→ The supervised training is just behavior cloning, which relies on the IID assumption

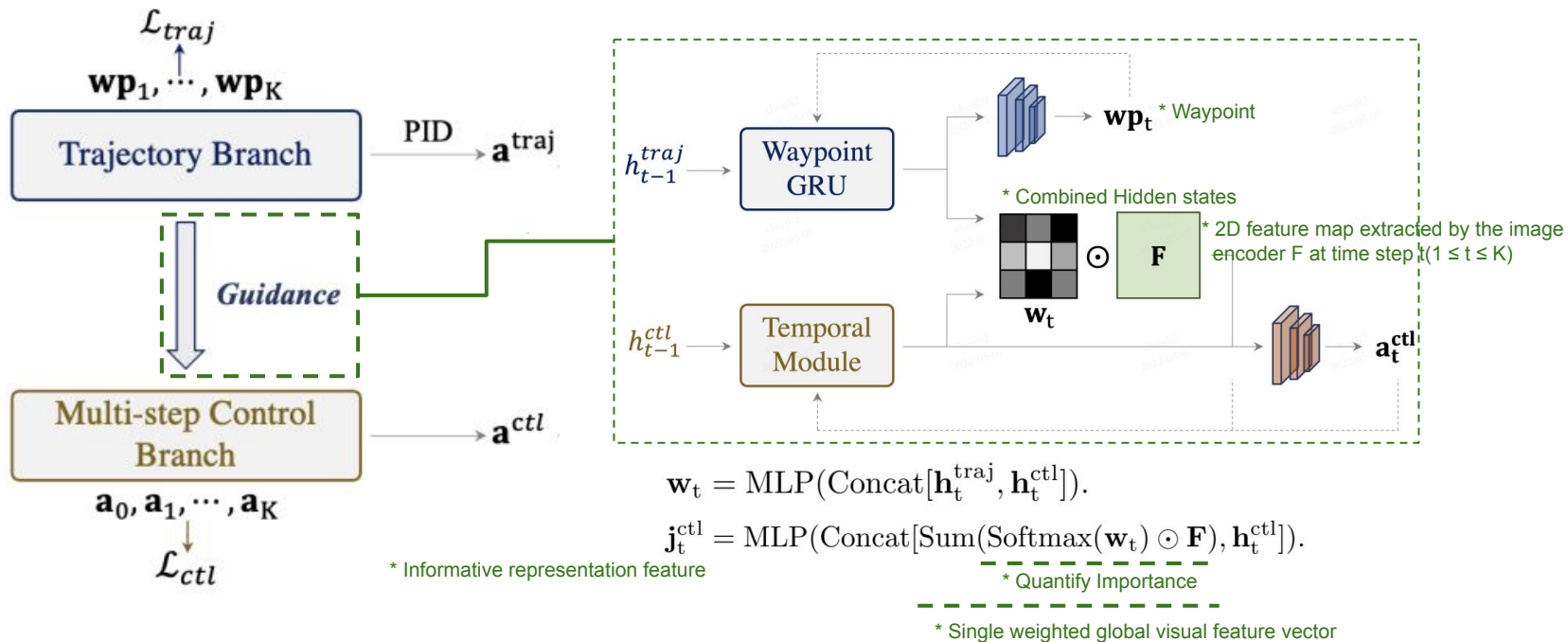
The distribution shifts in test cases, where the historical actions affect the future
→ Temporal module(e.g. GRU) use instead of RL which relies on Markov Decision Process

Method

TCP(Trajectory-guided Control Prediction)

: Trajectory-guided attention

- In this way, the **spatial attention is guided by trajectory-based priors**, providing a strong structural inductive bias to the model.



Method

TCP(Trajectory-guided Control Prediction)

: Loss function

$$\mathcal{L} = \lambda_{traj} \cdot \mathcal{L}_{traj} + \lambda_{ctl} \cdot \mathcal{L}_{ctl} + \lambda_{aux} \cdot \mathcal{L}_{aux}.$$

$$\mathcal{L}_{traj} = \sum_{t=1}^K \|\mathbf{w}\mathbf{p}_t - \hat{\mathbf{w}}\mathbf{p}_t\|_1 + \lambda_F \cdot \mathcal{L}_F(\mathbf{j}_0^{traj}, \mathbf{j}_0^{Expert})$$

* L1 loss from ground-truth trajectory

* Indicates the feature loss measuring the L2 distance

$$\mathcal{L}_{ctl} = \underbrace{KL(\text{Beta}(a_0) \parallel \text{Beta}(\hat{a}_0))}_{\text{현재 step의 KL}} + \frac{1}{K} \sum_{t=1}^K \underbrace{KL(\text{Beta}(a_t) \parallel \text{Beta}(\hat{a}_t))}_{\text{미래 step들의 KL}}$$
$$+ \lambda_F \cdot \underbrace{\mathcal{L}_F(j_0^{ctl}, j_0^{Expert})}_{\text{현재 feature loss}} + \frac{1}{K} \sum_{t=1}^K \underbrace{\mathcal{L}_F(j_t^{ctl}, j_t^{Expert})}_{\text{미래 feature loss}}$$

$$\mathcal{L}_{aux} = \lambda_1 \cdot \|\hat{s} - s^*\|_1 + \lambda_2 \cdot \|\hat{v} - v^*\|_2^2$$

* Speed/value prediction head added to Latent feature

Method

TCP(Trajectory-guided Control Prediction)

: Output Fusion

Algorithm 1: Situation based fusion scheme to combine the two output paradigms

Input: sensory input $\mathbf{i}$, speed of the ego vehicle v ,
high level navigation information $\mathbf{g}$.

Hyper parameters : combination weight $\alpha \in [0, 0.5]$

Output: final control signals $\mathbf{a}$

$\{\mathbf{wp}_t\}_{t=0}^K, \mathbf{a}^{\text{ctl}} \leftarrow \text{TCP}(\mathbf{i}, v, \mathbf{g})$

$\mathbf{a}^{\text{traj}} \leftarrow \text{Low-level Controller}(\{\mathbf{wp}_t\}_{t=0}^K)$

Get current *situation*

if *situation* is trajectory specialized **then**

$\mathbf{a} \leftarrow \alpha \times \mathbf{a}^{\text{ctl}} + (1 - \alpha) \times \mathbf{a}^{\text{traj}}$

else

$\mathbf{a} \leftarrow \alpha \times \mathbf{a}^{\text{traj}} + (1 - \alpha) \times \mathbf{a}^{\text{ctl}}$

end

* In our experiment, we choose the situation according to whether the ego vehicle is turning, implying that if it is turning, the situation is control specialized otherwise trajectory specialized.

Experiments

Setup

: Task & Evaluation Metrics

- Task Definition
 - : Closed-loop driving task with sparse navigation points + high-level commands
 - : Commands: straight, turn left/right, lane changing, lane following
- Challenging Scenarios
 - : Obstacle avoidance, Crossing unsignalized intersection, Sudden control loss
- Objective
 - : Autonomous agent drives towards destination point in realistic traffic situations

: Three Major Evaluation Metrics

- Driving Score
 - : Main metric = Route Completion $\times$ Infraction Score
- Route Completion
 - : Percentage of route completed by autonomous agent
- Infraction Score
 - : Measures infractions with pedestrians, vehicles, road layouts, red lights, etc.

Experiments

State-of-the-art Comparison

: Method Variants Tested

- TCP: Base model with shared encoders
- TCP-SB: Separate encoders for two branches (vs shared in TCP)
- TCP-Ens: Ensemble of TCP and TCP-SB : Measures infractions with pedestrians, vehicles, road layouts, red lights, etc.

Rank	Method	Sensor Inputs		Key Metrics ↑		
		#Cameras	LiDAR	Driving Score	Route Completion	Infraction Score
1	TCP-Ens (ours)	1	✗	75.137	85.629	0.873
1	TCP (ours)	1	✗	69.714	82.962	0.851
1	TCP-SB (ours)	1	✗	68.695	82.957	0.833
2	LAV [10]	4	✓	61.846	94.459	0.640
3	Transfuser	3	✓	61.181	86.694	0.714
4	Latent Transfuser	3	✗	45.029	75.366	0.618
5	GRIAD [9]	3	✗	36.787	61.855	0.597
6	Transfuser+ [29]	4	✓	34.577	69.841	0.562
7	WoR [12]	4	✗	31.370	57.647	0.557
8	MaRLn [50]	1	✗	24.980	46.968	0.518
9	NEAT [15]	3	✗	21.832	41.707	0.650

Experiments

Control vs. Trajectory

: Shared Architecture Components:

- Visual Encoder: ResNet-34 for visual inputs
- Measurement Module: Encodes navigation information
- Auxiliary Tasks: Speed and value heads to help model better encode environment

Two Model Configurations:

- Control-Only: Predict control distribution based on concatenated latent feature from two encoders
- Trajectory-Only: Feed feature to GRU decoder to generate waypoints

Model	Driving Score	Collisions vehicles		Collisions layout		Off-road infractions		Agent blocked	
		#/km ↓	TurnRatio	#/km ↓	TurnRatio	#/km ↓	TurnRatio	#/km ↓	TurnRatio
Control-Only	32.45±2.23	1.25	50.90%	0.23	10.00%	0.59	46.15%	0.41	50.00%
Trajectory-Only	28.29±3.03	0.85	38.70%	0.77	64.20%	0.74	62.90%	0.77	64.20%

Experiments

Ablative Study and Visualization

: Component analysis

Exp.	Driving Score	Route Completion	Infraction Score
Control	32.45±2.23	76.54±3.22	0.45±0.03
+ traj-task	34.98±1.96	81.32±5.50	0.49±0.05
+ temporal	42.87±4.77	87.51 ±3.63	0.49±0.07
+ traj-attn	46.08±3.47	84.95±1.84	0.56±0.03
+ fusion	57.01 ±1.88	85.27±1.20	0.67 ±0.01

Progressive Component Validation

- Experimental Design: Validate effectiveness of trajectory-guided multi-step control prediction design
- Ablation Protocol: Only employ control branch output except for final fusion application
- Adding a trajectory branch as Trajectory auxiliary tasks: + 2.5
- adding the multi-step predictions with our temporal module: + 7.9
- adding the trajectory-guided attention: + 3.2

: Multi-task vs. Ensemble

Exp.	Driving Score	#Param.	FLOPs	FPS
Ensemble	45.03±1.28	46.81M	17.07G	69.47
MTL	48.27±0.58	23.58M	8.54G	133.30
TCP-SB	52.46±4.66	47.26M	17.07G	69.35
TCP	57.01±1.88	25.77M	8.54G	125.71
TCP-Ens	59.09±3.66	73.03M	25.61G	44.70

Method

- Ensemble: Directly combining Control-Only and Trajectory-Only outputs with situation-based fusion
- MTL : Shared CNN backbone + measurement encoders
→ trajectory & control branches * Control branch w/o traj guide
- TCP-Ens : Conventional ensemble combining TCP and TCP-SB results

Thank you.